

산업공학과 머신러닝 서버 – 주요 사양

- 제조사 및 모델: **BARO AI / POSEIDON H 2000 Alpha**

구분	항목	사양
Hardware	CPU	INTEL i9 X-Processor (10 core, 20 thread, Max 4.7GHz, Base 3.7GHz)
	GPU	NVIDIA Geforce RTX 4090 24GB × 2 (16,384CUDA core, 512 Tensor core, 128 RT Core, 1,008GB/s Memory Bandwidth)
	RAM	DDR4 32GB (PC4-25600) 3.2GHz Clock
	Storage	NVMe SSD 1TB
Software	OS	Ubuntu 22.04
	Library	• CUDA 11.8, cuDNN (8.6.0), NCCL (2.15.5) • Anaconda 3 • PostgreSQL • Docker

산업공학과 머신러닝 서버 – 사용 방법

- 머신러닝 서버 사용이 필요한 학생은 아래의 절차에 따라 사용 승인 후 사용이 가능합니다.

1. 관리 담당 교수님께 아래 신청 사항을 작성하여 이메일로 제출

- 학번 및 이름 – 산업공학과 학부생 및 대학원생만 가능함.
- 사용 목적 및 기간 – 교과목 내 프로젝트, 경진대회 등 고사양 머신러닝 모델 학습 및 실행 환경이 필요한 경우에 한함.
- 사용 계획 – 프로젝트, 경진대회 등에 필요한 활용 계획(첨부 양식 사용)을 제출함.
- 제출 이메일: 김형중 조교수, hyungjungkim@konkuk.ac.kr

2. 신청 내용 확인 후 승인 또는 반려 이메일 회신

- 일주일 이내에 이메일로 회신함.
- 승인 시, 서버 관리 담당자가 접속 정보(아이디 및 비밀번호)를 제공함.

3. 사용 시, 도커(docker)를 사용하여 별도의 공간에서 자유롭게 사용

- 머신러닝 서버는 리눅스 OS로 운영되며, SSH 원격 접속을 통해서 사용함. 접속 정보와 함께, 간단한 사용 방법을 안내 예정이나, 리눅스 OS 및 머신러닝 라이브러리 사용 방법은 제공되지 않음.
- 사용 기간 종료 후 계정을 삭제하므로 중요한 데이터는 사용자가 사전에 백업을 해야함.

1 서버 원격 접속.

SSH란?

SSH 접속에 필요한
정보

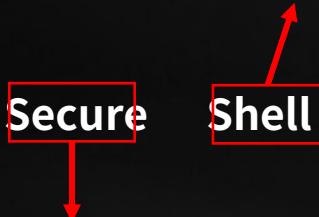
SSH 접속 방법
(CLI/GUI)

SSH를 이용한 원격 접속

SSH란?

Local PC에서 원격지 서버로 CLI로 접속

Secure **Shell**



접속에 필요한 정보를 암호화하여 보안성 향상

서버 원격 접속

SSH 접속에 필요한 정보

서버 내 SSH Server 설치 여부
(openssh-server)

서버의 IP

서버의 SSH Server Port

서버에 생성된 사용자 ID / PW(Key File)

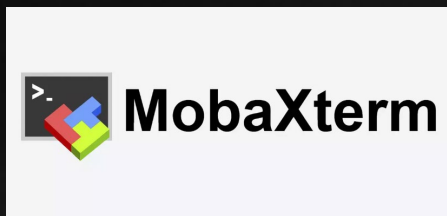
SSH를 이용한 원격 접속

SSH 접속 방법(CLI/GUI)

Terminal



GUI



SSH를 이용한 원격 접속

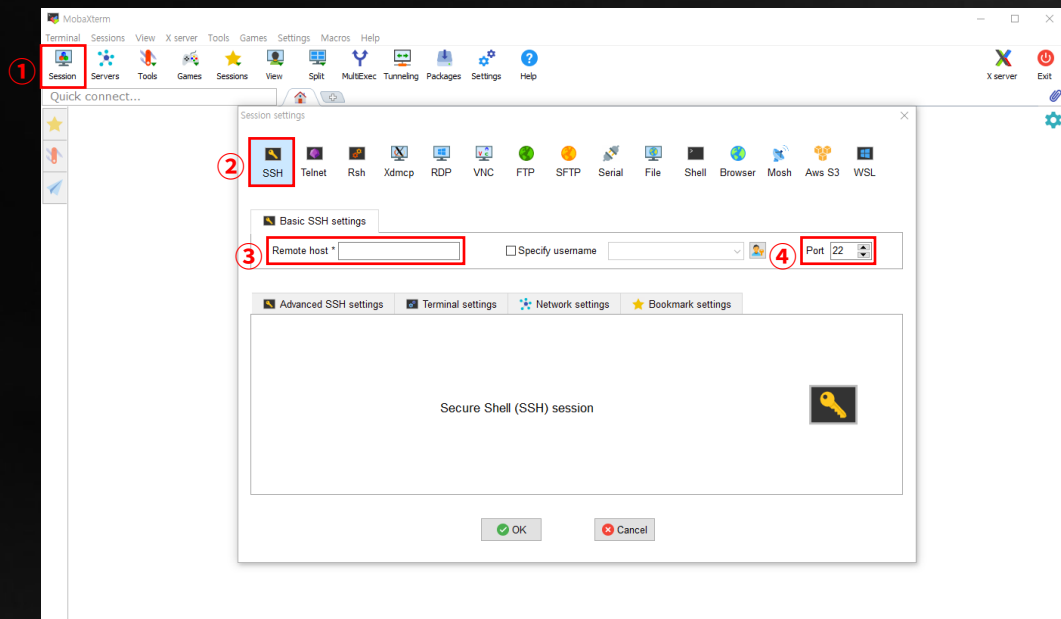
SSH 접속 방법(CLI)

\$ ssh [계정명]@[원격지 서버의 IP] -p [원격지 서버의 SSH Port]

```
(base) oem@kyle-vmware:~$ ssh oem@192.168.0.15 -p 7722
The authenticity of host '[192.168.0.15]:7722 ([192.168.0.15]:7722)' can't be established.
ECDSA key fingerprint is SHA256:5P7WNqKFtZuZERTISpBbcp2A0fG58cRuC2Eww+N9jy0.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.0.15]:7722' (ECDSA) to the list of known hosts.
oem@192.168.0.15's password: █
```

SSH를 이용한 원격 접속

SSH 접속 방법(GUI)



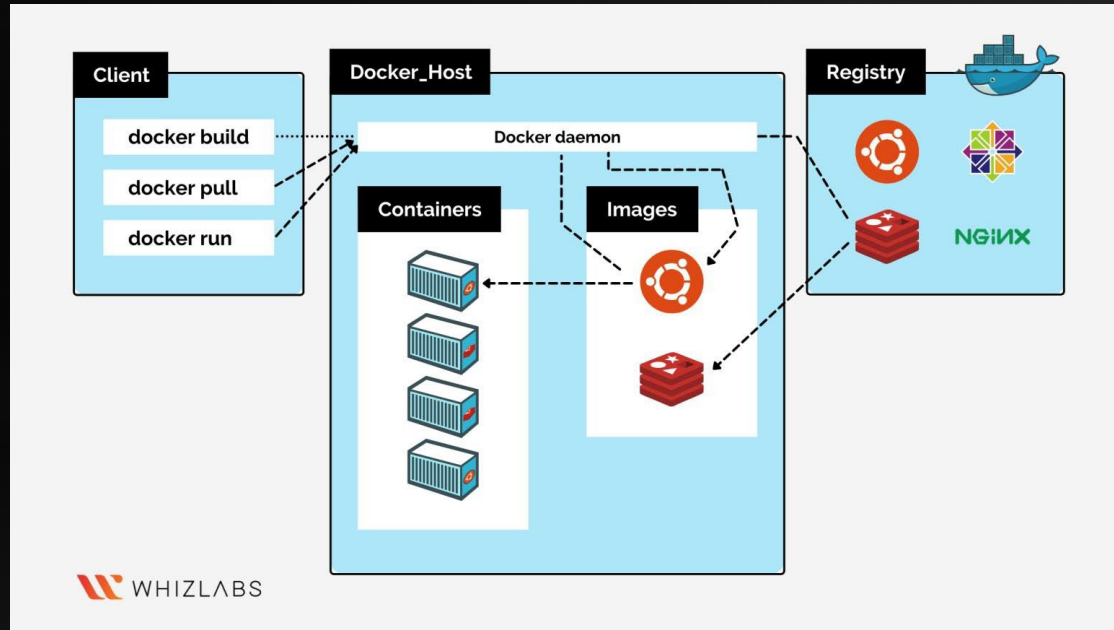
MobaXterm 설치 링크

<https://mobaxterm.mobatek.net/download-home-edition.html>
(Portable Edition 설치)

2 Docker Engine.

- Docker Architecture
- Docker Container Operations
- GPU Allocation
- Host Machine Communication
- Example

2-1 Docker Architecture



2-2 Docker Container Operations

List Containers

- 현재 활성화 되거나 중지된 컨테이너의 목록을 보는 명령어입니다.

명령어: `$ docker ps` (현재 실행중인 컨테이너 목록)

명령어: `$ docker ps -a` (모든 컨테이너 목록)

```
(base) oem@oem-MS73-HB0-000:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
(base) oem@oem-MS73-HB0-000:~\$						

List Images

- 로컬 저장소에서 Pull받은 Image목록을 보여줍니다.

명령어: `$ docker images` (현재 Tag및 버전이 명시된 이미지)

명령어: `$ docker images -a` (모든 이미지)

```
(base) oem@oem-MS73-HB0-000:~$ docker images -a
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    9c7a54a9a43c   4 months ago   13.3kB
(base) oem@oem-MS73-HB0-000:~$
```

2-2 Docker Container Operations

Pull Images

- Public Repository 혹은 Private Repository에 있는 Container이미지를 Local저장소로 다운로드 받습니다.

명령어: `$ docker pull [Options] Name[:Tag[@DIGEST]`

Ex) `$docker pull ubuntu:20.04`



```
oem@oem-virtual-machine:~$ docker pull ubuntu:20.04
20.04: Pulling from library/ubuntu
edaedc954fb5: Pull complete
Digest: sha256:33a5cc25d22c45900796a1aca487ad7a7cb09f09ea00b779e3b2026b4fc2faba
Status: Downloaded newer image for ubuntu:20.04
docker.io/library/ubuntu:20.04
oem@oem-virtual-machine:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        20.04     6df894023726   5 weeks ago    72.8MB
oem@oem-virtual-machine:~$
```

2-2 Docker Container Operations

Remove Images

- 이미지를 삭제하는 명령어는 아래와 같습니다.

명령어: `$ docker rmi [options] Image명:버전명`

Ex) `$ docker rmi ubuntu:20.04`

```
oem@oem-virtual-machine:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        20.04     6df894023726   5 weeks ago    72.8MB
oem@oem-virtual-machine:~$ docker rmi ubuntu:20.04
Untagged: ubuntu:20.04
Untagged: ubuntu@sha256:33a5cc25d22c45900796a1aca487ad7a7cb09f09ea00b779e3b2026b4fc2faba
Deleted: sha256:6df89402372646d400cf092016c28066391a26f5d46c00b1153e75003465484d
Deleted: sha256:954c82bdeb5fcc80094317528fa3fcbb1026aeff64f872527d35ec9b4343b84d
oem@oem-virtual-machine:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
oem@oem-virtual-machine:~$
```

2-2 Docker Container Operations

Run Containers

- Container를 실행하는 명령어입니다.

명령어 : `$ docker run [Options] [Image] [Command]`

Ex) `$ docker run -itd nginx:latest /bin/sh`

```
oem@oem-virtual-machine:~$ docker run -itd nginx:latest /bin/sh
4c943ede67b7d565959790ce9dc5f46ae4bac9e1c2101be4f0d0f7905625462d
oem@oem-virtual-machine:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
4c943ede67b7   nginx:latest  "/docker-entrypoint..."  2 seconds ago  Up 1 second   80/tcp       thirsty_lamarr
oem@oem-virtual-machine:~$
```

Execute Containers

- 실행중인 Container에 접속 및 명령어를 전달을 위한 명령어입니다.

명령어 : `$ docker exec [Options] [Container ID/Name] [Command]`

Ex) `$ docker exec -it thirsty_lamarr /bin/sh`

```
oem@oem-virtual-machine:~$ docker exec -it thirsty_lamarr /bin/sh
#
```

Detach = Ctrl + p + q

Attach = `docker attach` [컨테이너명]

2-2 Docker Container Operations

Start, Stop, Restart Containers

- Container의 상태를 제어하기 위한 명령어 입니다.

명령어 : `$ docker start/stop/restart [Options] [Container ID/Name]`

Ex) `$docker stop thirsty_lamarr`

```
oem@oem-virtual-machine:~$ docker stop thirsty_lamarr
thirsty_lamarr
```

Remove Containers

- 정지중인 Container를 삭제하기 위한 명령어 입니다.

명령어 : `$ docker rm [Options] [Container ID/Name]`

Ex) `$docker rm thirsty_lamarr`

```
oem@oem-virtual-machine:~$ docker rm thirsty_lamarr
thirsty_lamarr
oem@oem-virtual-machine:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
oem@oem-virtual-machine:~$
```

2-3 GPU Allocation

GPU Device 방식

- NVIDIA-Docker 사용시 device옵션을 이용하여 GPU를 선택할 수 있습니다.
- 명령어: `$ docker run [Options] -gpus "device=[사용할 GPU 번호]" [Image] [Command]`
Ex) `$docker run --it "device=0,1" nvidia/cuda:10.0-cudnn7-devel-ubuntu18.04 nvidia-smi`

```
(base) oem@tie1:~$ docker run -it --gpus '"device=0,1,2,3"' nvidia/cuda:10.0-cudnn7-devel-ubuntu18.04 nvidia-smi
Tue Jul 27 06:56:59 2021
```

NVIDIA-SMI 460.73.01 Driver Version: 460.73.01 CUDA Version: 11.2									
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC			
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.			
						MIG M.			
0	GeForce RTX 3090	Off	00000000:19:00.0	Off	0%	N/A			
0%	25C	P8 30W / 350W	10MiB / 24268MiB			Default			
						N/A			
1	GeForce RTX 3090	Off	00000000:1A:00.0	Off	0%	N/A			
0%	25C	P8 24W / 350W	10MiB / 24268MiB			Default			
						N/A			
2	GeForce RTX 3090	Off	00000000:67:00.0	Off	0%	N/A			
0%	26C	P8 35W / 350W	10MiB / 24268MiB			Default			
						N/A			
3	GeForce RTX 3090	Off	00000000:68:00.0	On	0%	N/A			
0%	25C	P8 26W / 350W	699MiB / 24267MiB			Default			
						N/A			

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
ID	ID	ID				
=====						

2-3 Host machine communication

File copy

- Docker container 내부의 파일을 복사하기 위해 아래의 명령어를 통해 Container to Host, Host to Container로 데이터를 복사할 수 있습니다.
- 명령어 : `$ docker cp [복사할 로컬 데이터] [컨테이너명]:[컨테이너 내부 경로] Host -> Container의 경우`
- 명령어 : `$ docker cp [컨테이너명]:[컨테이너 데이터 경로] [Host 복사경로] Container -> Host의 경우`
Ex) `$ docker cp test.txt jovial_Bhabha:/`
- Ex) `$ docker cp jovial_Bhabha:/container.txt ./`

Volume Mount

- Host에서 사용하는 특정 Directory를 Docker Container와 공유할 수 있습니다.
- 명령어 : `$ docker run [Options] -v [Host 폴더]:[Container에 생성할 폴더 이름] [Image] [Command]`
Ex) `$ docker run -itd -v ./test:/data nginx:latest /bin/bash`

2-3 Host machine communication

Port Forwarding

- Docker container로 Host의 Port를 mapping할 수 있습니다.
- 명령어 : `$ docker run [Options] -p [Host의 Port]:[Container Port] [Image] [Command]`
Ex) `$ docker run -itd -p 7777:80 nginx:latest`

Host Network

- Host에서 사용하는 모든 포트를 Docker Container에서 점유합니다.
- 명령어 : `$ docker run [Options] --network=host [Image] [Command]`
Ex) `$ docker run -itd --network=host nginx:latest`